

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Announcements

- **Exercise 3 solutions** have been posted.
- The **Final Project Requirement** document (ics604-S26-final_project.pdf) is now available in the *Resources* module on Lamaku.
 - Please note that there have been updates to the project requirements. The revised syllabus has been posted on Lamaku.
- **Participation grades** are available on Lamaku.
- No class on Monday, February 16 (Presidents' Day).

Reminder: Homework Assignment 1

- Due: **11:59 PM today** (Wednesday, February 11)
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Do not upload the data file.
 - Make sure to **rename your notebook file** as instructed.

Lecture 8

- Plotting and Visualization Using Matplotlib
- Random Number Generation in Python and Common Plot Types

Plotting and visualization using Matplotlib

- Making plots is one of the most critical tasks in data science.
- Matplotlib is a desktop package for creating primarily 2D publication-ready plots.
- Inspired by MATLAB and commands almost equivalent across MATLAB and Matplotlib.
- Matplotlib has spawned many add-on toolkits for data visualization that build on top of Matplotlib.

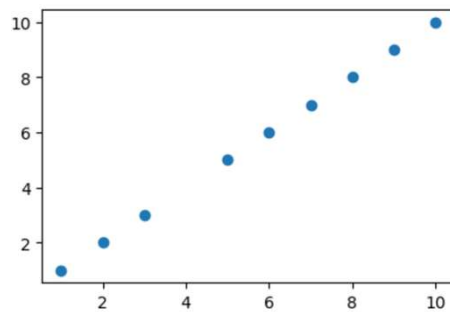
Matplotlib: A brief intro

- Matplotlib can be overwhelming, a good place to start is the gallery. (<https://matplotlib.org/stable/gallery/index.html>)
 - Showcases and includes code examples for various plot types.
- Supports a wide array of plots, from simple line charts to more complex visualizations.
- Matplotlib's main module is pyplot.
 - Imported using the `plt` alias:

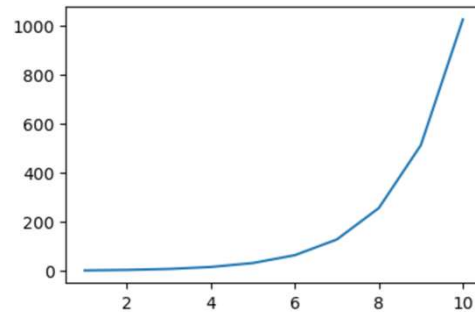
```
import matplotlib.pyplot as plt
```

Plotting with X and Y axes

```
import matplotlib.pyplot as plt  
  
x = [1, 2, 3, 5, 6, 7, 8, 9, 10]  
y = [1, 2, 3, 5, 6, 7, 8, 9, 10]  
  
plt.scatter(x, y)  
plt.show()
```

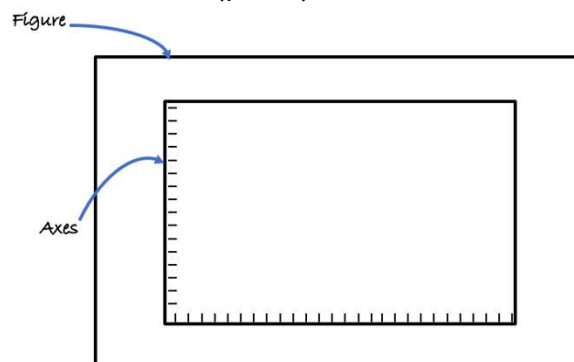


```
x = np.arange(1, 11)  
y = [2**i for i in x]  
  
plt.plot(x, y)  
plt.show()
```



Plot figure and axes

- Plots in Matplotlib reside within a Figure object.
- A figure contains the overall window where plotting happens.
 - Contains the title, axes (plots), labels, ticks, etc.



Plot figure and axes (cont'd.)

- Figures are empty by default. Therefore, you need to add an axes (plot) to visualize your data.

```
fig = plt.figure()
```

```
<Figure size 640x480 with 0 Axes>
```

- To add a plot, you need to determine the geometry and indicate where you want the plot to go.

- This returns the axes of a new subplot. It's an object of type Axes.

```
fig.add_axes(rect) or fig.add_subplot()
```

- Therefore, figure and plot/subplot (axes) are two different things.

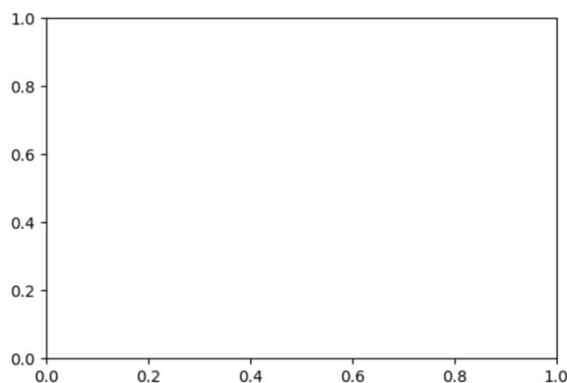
```
fig = plt.figure()
```

```
# add_axes takes a rectangle [x0, y0, width, height]
```

```
ax = fig.add_axes([0, 0, 1, 1])
```

```
print(f"ax is of type {type(ax)}")
```

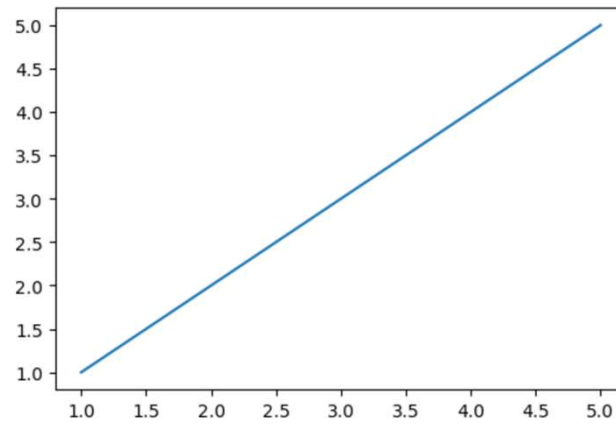
```
ax is of type <class 'matplotlib.axes._axes.Axes'>
```



```
fig = plt.figure()

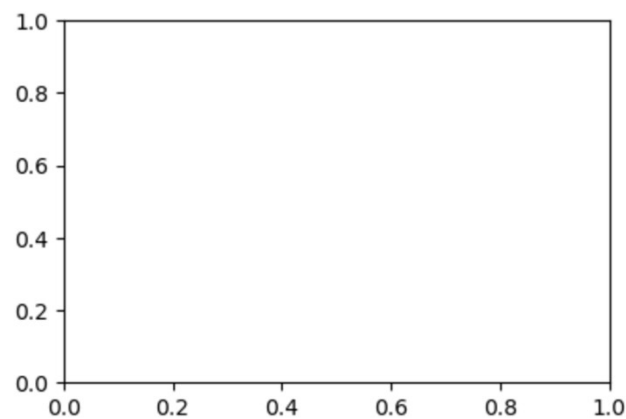
ax = fig.add_axes([0, 0, 1, 1])
ax.plot([1, 2, 3, 4, 5], [1, 2, 3, 4, 5])

plt.show()
```



```
fig = plt.figure()
ax = fig.add_subplot()
print(f"ax is of type {type(ax)}")
```

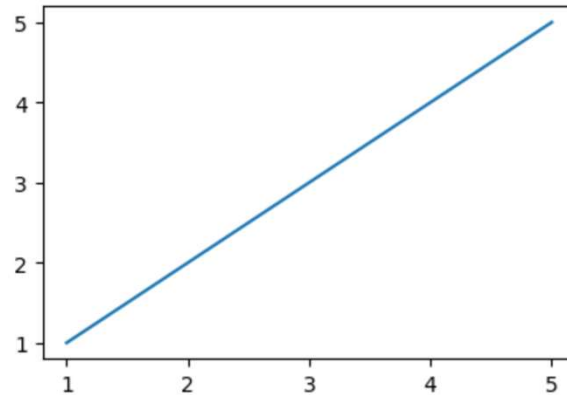
ax is of type <class 'matplotlib.axes._axes.Axes'>



```
fig = plt.figure()

ax = fig.add_subplot()
ax.plot([1, 2, 3, 4, 5], [1, 2, 3, 4, 5])

plt.show()
```



Plotting steps

- When you issue the command `plt.plot(x,y)`, the following happens.
 - Matplotlib uses the figure created in a cell or creates one if none exists.
 - It uses the last axes used or creates one if necessary.

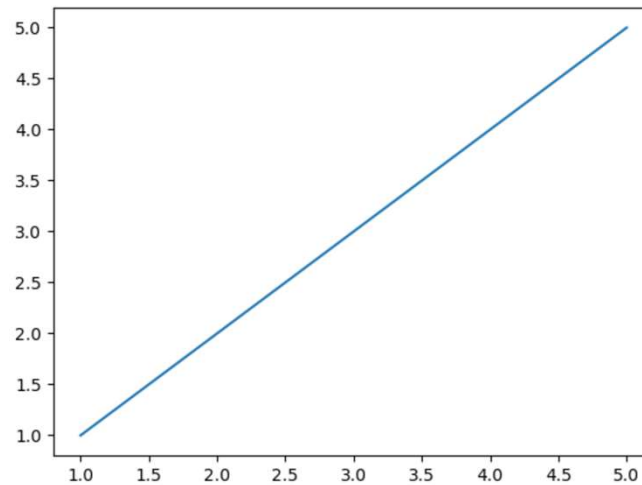
```
{ fig = plt.figure()
  ax = fig.add_subplot()
  ax.plot([1, 2, 3, 4, 5], [1, 2, 3, 4, 5])
```

- Is equivalent to

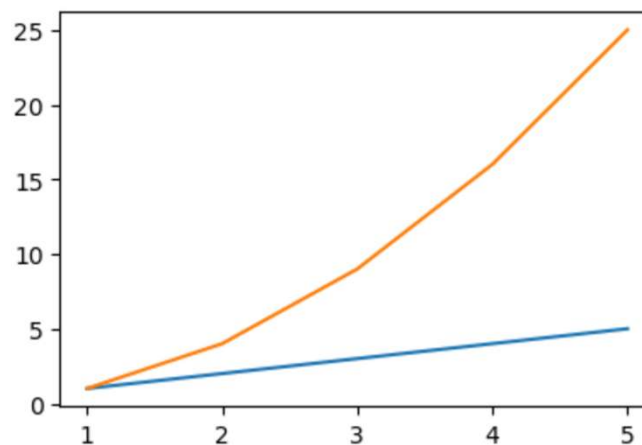
```
plt.plot([1, 2, 3, 4, 5], [1, 2, 3, 4, 5])
```

```
# When calling plt.plot() without explicitly creating a figure or adding axes,  
# Matplotlib automatically creates a default figure and axes.
```

```
plt.plot([1, 2, 3, 4, 5], [1, 2, 3, 4, 5])  
plt.show()
```



```
# automatically plot on any created figure and axis  
plt.plot([1, 2, 3, 4, 5], [1, 2, 3, 4, 5])  
plt.plot([1, 2, 3, 4, 5], [1, 4, 9, 16, 25])  
plt.show()
```



Creating multiple subplots

- It is possible to draw multiple plots (subplots) in one figure using the `plt.subplots()`, `plt.subplot()`, or `figure.add_subplot()` functions.
 - Each subplot has its own set of axes.
- Subplot layout is organized in rows and columns.
 - A subplot needs to know where in the row/column layout it belongs.
`fig.add_subplot(num_rows, num_cols, position_of_subplot)`

*add a subplot
each time*

```
# Sample data
x = [1, 2, 3, 4, 5]
y_1 = [1, 2, 3, 4, 5]
y_2 = [i**2 for i in x]
y_3 = [i**3 for i in x]
y_4 = [i**4 for i in x]

x, y_1, y_2, y_3, y_4

([1, 2, 3, 4, 5],
 [1, 2, 3, 4, 5],
 [1, 4, 9, 16, 25],
 [1, 8, 27, 64, 125],
 [1, 16, 81, 256, 625])
```

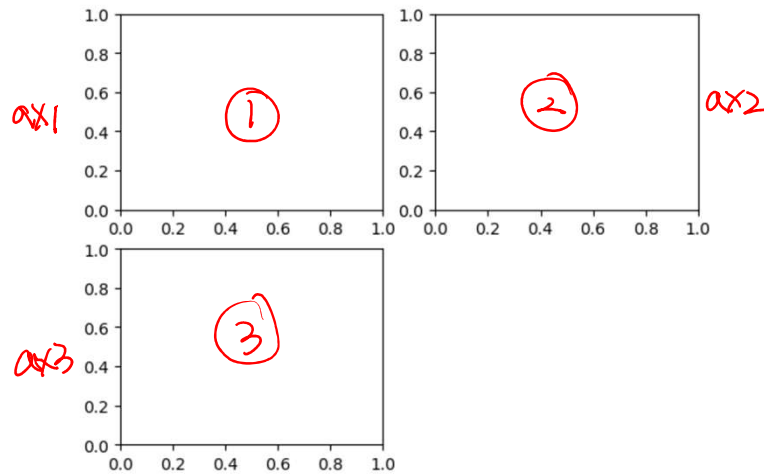
```
# creating the subplots
```

```
fig = plt.figure()
```

```
ax1 = fig.add_subplot(2, 2, 1) # equivalent to ax1 = fig.add_subplot(221)
```

```
ax2 = fig.add_subplot(2, 2, 2) # equivalent to ax2 = fig.add_subplot(222)
```

```
ax3 = fig.add_subplot(2, 2, 3) # equivalent to ax3 = fig.add_subplot(223)
```



```
fig = plt.figure()
```

```
ax1 = fig.add_subplot(2, 2, 1) # equivalent to ax1 = fig.add_subplot(221)
```

```
ax2 = fig.add_subplot(2, 2, 2) # equivalent to ax2 = fig.add_subplot(222)
```

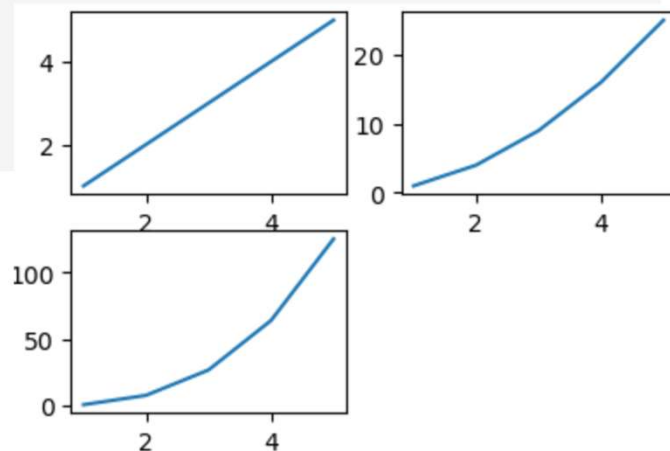
```
ax3 = fig.add_subplot(2, 2, 3) # equivalent to ax3 = fig.add_subplot(223)
```

```
ax1.plot(x, y_1)
```

```
ax2.plot(x, y_2)
```

```
ax3.plot(x, y_3)
```

```
plt.show()
```



```
fig = plt.figure()
```

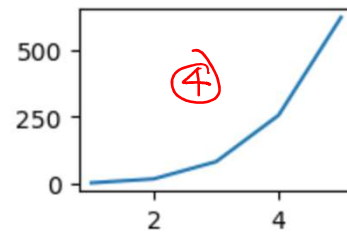
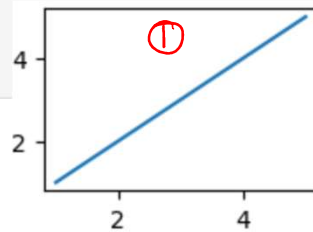
```
ax1 = fig.add_subplot(2, 2, 1) # equivalent to ax1 = fig.add_subplot(221)
```

```
ax2 = fig.add_subplot(2, 2, 4) # equivalent to ax2 = fig.add_subplot(224)
```

```
ax1.plot(x, y_1)
```

```
ax2.plot(x, y_4)
```

```
plt.show()
```



plot uses the last plot used.

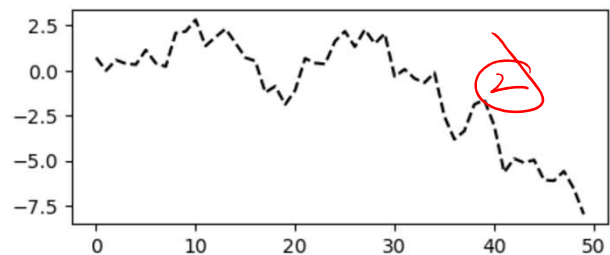
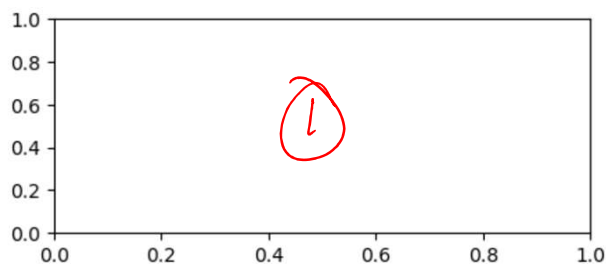
```
fig = plt.figure(figsize=(11, 2))
```

```
ax1 = fig.add_subplot(1, 2, 1) # equivalent to ax1 = fig.add_subplot(121)
```

```
ax2 = fig.add_subplot(1, 2, 2) # equivalent to ax2 = fig.add_subplot(122)
```

```
plt.plot(np.random.randn(50).cumsum(), 'k--') ⇔ ax2.plot( . . . )
```

```
plt.show()
```



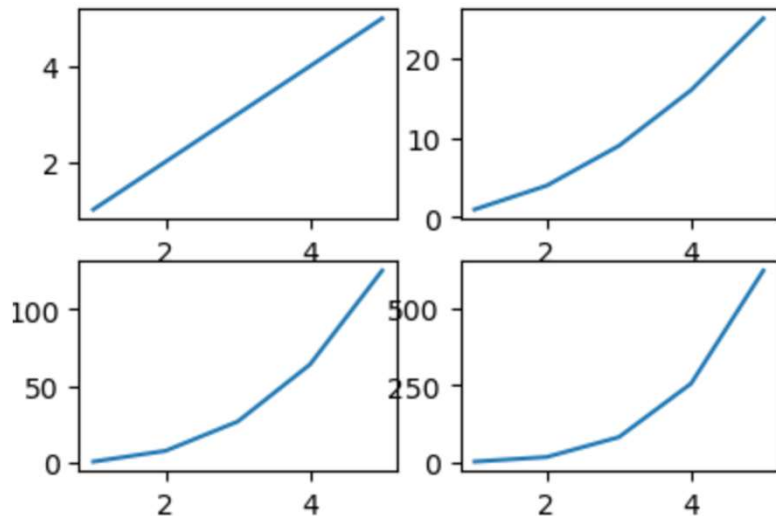
```
# Using subplot()
```

```
plt.subplot(2, 2, 1)  
plt.plot(x, y_1)
```

```
plt.subplot(2, 2, 2)  
plt.plot(x, y_2)
```

```
plt.subplot(2, 2, 3)  
plt.plot(x, y_3)
```

```
plt.subplot(2, 2, 4)  
plt.plot(x, y_4)  
plt.show()
```



Creating an array of subplots

- Creating an array of subplots is a common task.
- `matplotlib` includes a function that generates the figure and all the plots using a single function.
- The plots can share the same x-axis or y-axis.
- Many other options to control the spacing, margins, figure size, etc.

```
fig, axes = plt.subplots(2, 3, figsize=(14, 6))
```

```
print(axes)
print(type(axes))
print(type(axes[0, 0]))
```

```
axes[0, 0].plot(x, y_1)
axes[1, 2].plot(x, y_4)
```

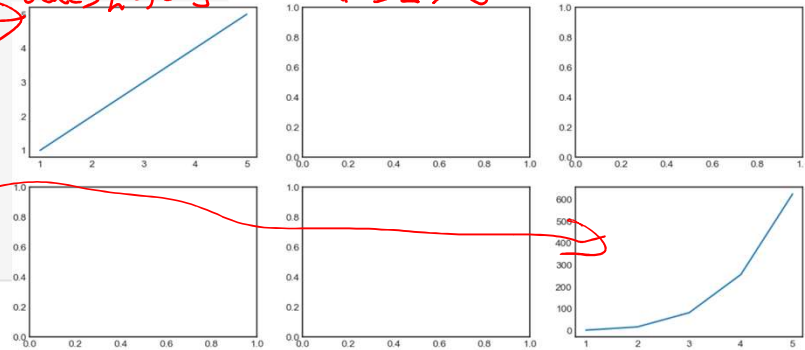
```
plt.show()
```

```
[[<Axes: > <Axes: > <Axes: >]
 [<Axes: > <Axes: > <Axes: >]]
<class 'numpy.ndarray'>
<class 'matplotlib.axes._axes.Axes'>
```

axes[0, 0]

axes[0, 1]

axes[1, 2]



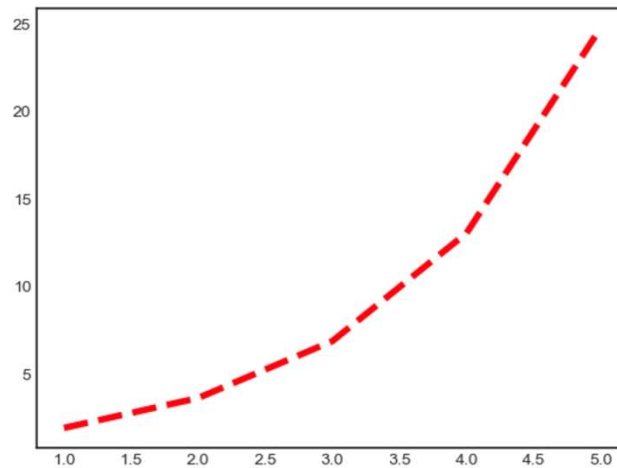
Basic plotting parameters

- Plots have various functions to customize them.
 - Options are either:
 - Figure-level
 - E.g., plot size, plot title, x-axis label, whitespace, legend, etc.
 - Plot-level:
 - E.g., line color, line type (e.g.: dashed, width=2), marker type, etc.
 - Example:

```
plt.plot(x, y, color='red', linestyle='--')
```

```
x = [1, 2, 3, 4, 5]
y = [1.9*i for i in x]

plt.plot(x, y, color='red', linestyle='--', linewidth=4)
plt.show()
```



Basic plotting parameters (cont'd.)

- In addition to the data, `plt.plot()` accepts a string abbreviating color and line style.

- This is a convenience.

```
plt.plot(x, y, 'rx')
```

```
plt.plot(x, y, 'k--')
```

- `plt.plot(x, y, 'k--')` can be expressed as `plt.plot(x, y, color='k', linestyle='--')`
- You can also specify color using hexadecimal code.

```

x = np.arange(10)

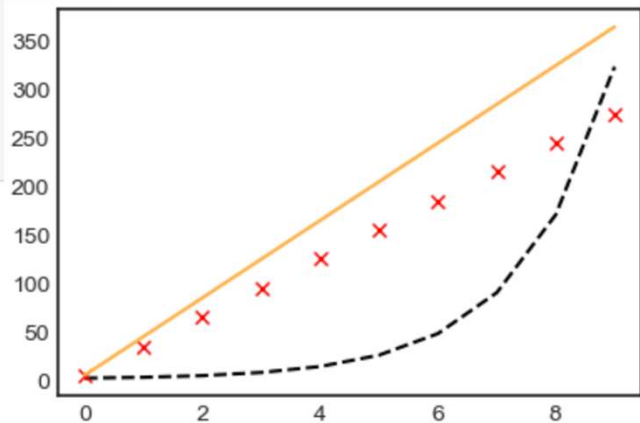
y = 1.9**x
plt.plot(x, y, 'k--') # black dashed line

y = 30*x + 4
plt.plot(x, y, 'rx') # red x markers

y = 40*x + 4
plt.plot(x, y, color='#feb24c')
plt.show()

```

w
 R B G
 [0, >55]



Plot legend

- Adding legends is critical for identifying plot elements.
- The easiest way to add a legend is to pass a label when adding each element and calling the `legend()` function on the plot.
 - The label can be a LaTeX format equation.
- The legend is placed on the plot in a way that interferes the least with the plot.
 - You can change the location of the legend manually.

```

x = [1, 2, 3, 4, 5]
y = [1.9*i for i in x]
plt.plot(x, y, 'k--', label="$1.9^x$")

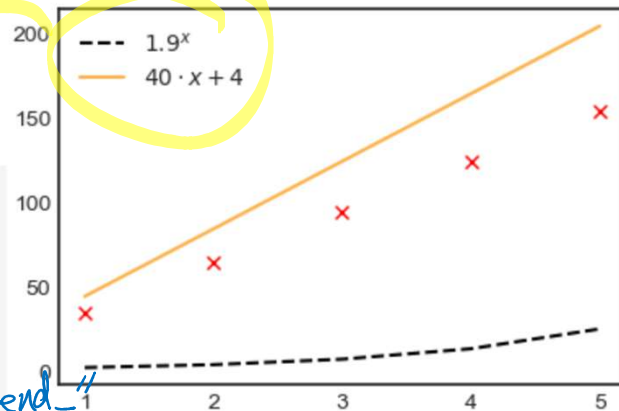
y = [30*i + 4 for i in x]
plt.plot(x, y, 'rx')

y = [40*i + 4 for i in x]
plt.plot(x, y, color='#feb24c', label="$40 \cdot x + 4$")

plt.legend()
plt.show()

```

label = "_no_legend_"



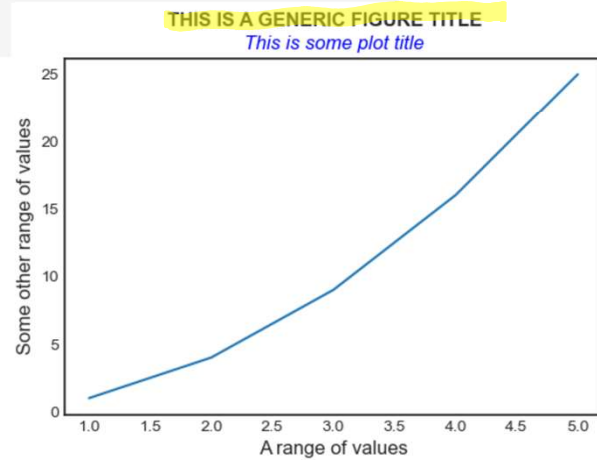
Plot labels

- The minimum required labels to describe a plot are meaningful x and y label, and a plot title.
- Axes can have a title, and so does the figure.
- Title and labels can be customized.
 - Change font type and size, color, etc.

```
fig = plt.figure(figsize=(6,4))
fig.suptitle('THIS IS A GENERIC FIGURE TITLE', fontweight="bold")

ax = fig.add_subplot()
ax.plot([1, 2, 3, 4, 5], [1, 4, 9, 16, 25])
ax.set_title("This is some plot title", color='blue', fontstyle="italic")
ax.set_xlabel("A range of values", fontsize=12)
ax.set_ylabel("Some other range of values", fontsize=12)

plt.show()
```



Changing style via presets

- Matplotlib provides various style presets to improve the look and feel of the plots.
 - The presets control the fonts, the colors, borders and line widths, etc.
- Example of applying a style preset:

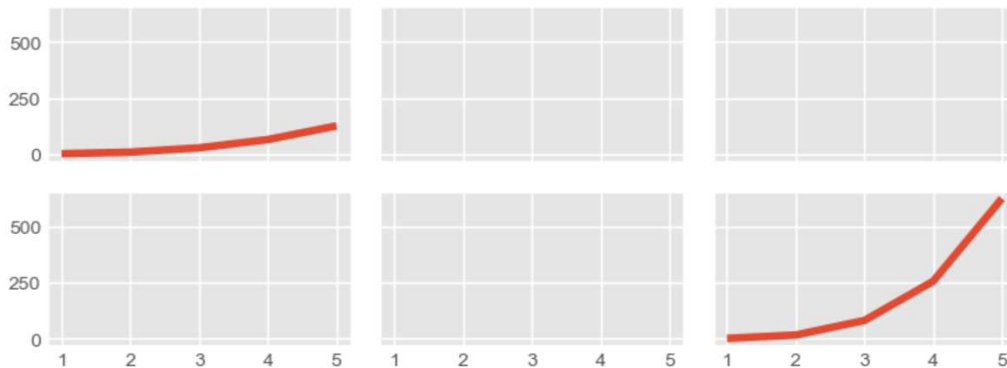
```
plt.style.use('seaborn-v0_8-white')
```

- Full list of available style presets at the [Matplotlib style sheets reference](#).

```
plt.style.use('ggplot')
fig, axes = plt.subplots(2, 3, sharex=True, sharey=True, figsize=(8, 3))

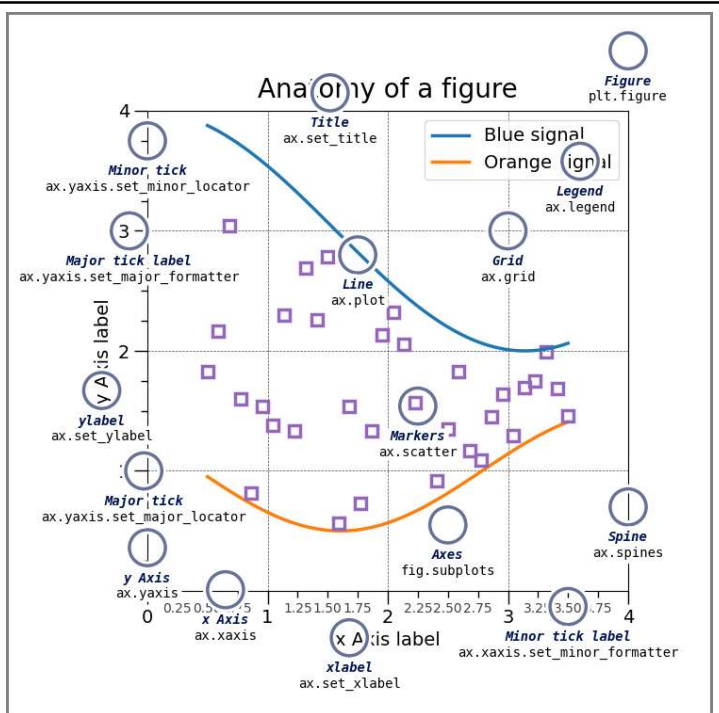
axes[0, 0].plot(x, y_3)
axes[1, 2].plot(x, y_4)

# Set whitespace between the subplots
plt.subplots_adjust(wspace=0.1, hspace=0.2)
plt.show()
```



Anatomy of a Matplotlib figure

- The properties of a matplotlib figure
(<https://matplotlib.org/stable/gallery/showcase/anatomy.html>)



Random Number Generation in Python and Common Plot Types

Random number generation

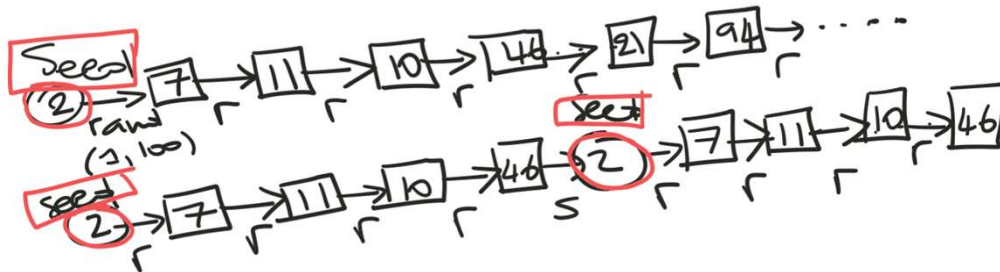
- Data we will generate throughout the course is **pseudorandom**.
 - Generated using an algorithm for generating seemingly random but still reproducible data.
 - They start with a random number, known as the seed, and then use an algorithm to generate a pseudo-random sequence of bits based on it.
- Should not be used for security purposes but is actually recommended for data science.
 - Reproducibility of random sequencing is convenient in experiments.

Random number generation (cont'd.)

- Basic random functionality is implemented in the random standard library.
- Many other sampling strategies (probability distributions) are implemented in numpy.
 - The same functions available in random are also available in numpy, although generally with more options.
 - numpy offer more flexible and efficient sampling strategies (beyond *uniform*).

(Pseudo)-random number generation and seeds

- The seed and the strategy determine the sequence of randomly generated numbers.
 - Suppose the strategy is *uniform* in range $[0, 100]$.



```
import random

random.seed(2)
random.randint(0, 100)
```

7

```
# Random sampling after setting a seed
random.seed(2)
rand_numbers = []
for i in range(10):
    rand_numbers.append(random.randint(0, 100))
print(rand_numbers)
```

[7, 11, 10, 46, 21, 94, 85, 39, 32, 77]

```
# Random sampling without setting a seed
rand_numbers = []
for i in range(20):
    rand_numbers.append(random.randint(0, 100))
print(rand_numbers)
```

[27, 77, 4, 74, 87, 20, 55, 81, 50, 92, 65, 47, 69, 56, 64, 34, 4, 3, 46, 59]

```
# Random sampling after resetting a seed
random.seed(2)
rand_numbers = []
for i in range(10):
    rand_numbers.append(random.randint(0, 100))
print(rand_numbers)
```

[7, 11, 10, 46, 21, 94, 85, 39, 32, 77]

Useful random functions

- **random.seed(s)**: Set the seed to s
- **random.randint(1, u)**: Generate a random integer in the range [1, u]
- **random.random()**: Generate a random value in the range [0, 1)
- **random.choice(items)**: Select a random element from the items
- **random.choices(items, k=n)**: Select n random elements from the items (*with replacement*)
- **random.sample(items, k=n)**: Sample $n \leq \text{len}(\text{items})$ random elements from the items (*without replacement*).
- **random.shuffle(items)**: Shuffle, *in-place*, the items in a pseudo random fashion

```
random.seed(42)
random.choice(["CS", "Math", "Eng", "Econ"])
```

```
'CS'
```

```
random.seed(42)
random.choices(["CS", "Math", "Eng", "Econ"], k=4)
```

```
['Eng', 'CS', 'Math', 'CS']
```

```
random.seed(42)
random.sample(["CS", "Math", "Eng", "Econ"], k=4)
```

```
['CS', 'Econ', 'Math', 'Eng']
```

```
random.seed(42)
random.choices(["CS", "Math", "Eng", "Econ"], k=5)
```

```
['Eng', 'CS', 'Math', 'CS', 'Eng']
```

```

random.seed(42)
random.sample(["CS", "Math", "Eng", "Econ"], k=5)

```

ValueError Traceback (most recent call last)

Cell In[10], line 2

```

1 random.seed(42)
----> 2 random.sample(["CS", "Math", "Eng", "Econ"], k=5)

```

File ~\miniconda3\envs\dsml\Lib\random.py:430, in Random.sample(self, population, k, counts)

```

428 randbelow = self._randbelow
429 if not 0 <= k <= n:
--> 430     raise ValueError("Sample larger than population or is negative")
431 result = [None] * k
432 setsize = 21 # size of a small set minus size of an empty list

```

ValueError: Sample larger than population or is negative

```

items = [1, 2, 3, 4, 5, 6]
random.shuffle(items)
print(items)

```

[4, 2, 3, 5, 1, 6]

Equivalent NumPy random functions

- **np.random.seed(s)**: Set the seed to s
- **np.random.randint(low, high, k)**: Generate k random integers in the range [low, high)
 - If high is None (default), then results are from [0, low)
 - k (output shape): integer or tuple of integers
- **np.random.rand(d0, d1, ..., dn)**: Generate an array of shape (d0, d1, ..., dn) of random values in the range [0, 1)

Equivalent NumPy random functions (cont'd.)

- **`np.random.choice(items, size=k, replace=True)`**: Select k random elements from the `items` with replacement by default
 - Obviates the need for `choices` and `sample` of random lib.
- **`np.random.shuffle(items)`**: Shuffle, in-place, `items` in a pseudo random fashion
- **`np.random.random(size)`** and **`np.random.random_sample(size)`** (or **`np.random.sample(size)`**) generate random values uniformly distributed in $[0, 1)$ with the specified shape, and are equivalent to **`np.random.rand(...)`**.
 - Note: `np.random.sample()` is not the same as Python's `random.sample()`

```
np.random.randint(0, 10, 10)
```

```
array([1, 8, 5, 1, 8, 2, 5, 1, 1, 3])
```

```
np.random.randint(5, None, 10)
```

```
array([1, 3, 2, 2, 0, 0, 2, 0, 1, 3])
```

```
np.random.randint(0, 10, (2, 5))
```

```
array([[4, 1, 5, 5, 1],  
       [0, 2, 6, 7, 6]])
```

```
np.random.rand(5)
```

```
array([0.99532271, 0.19332239, 0.45795339, 0.20922608, 0.28615908])
```

```
np.random.rand(2, 4)
```

```
array([[0.31842753, 0.40873321, 0.79130471, 0.09905721],  
       [0.75868803, 0.17835777, 0.80617843, 0.413422  ]])
```

rand, sample and random_sample have the same default behavior

```
np.random.seed(42)
print(np.random.random_sample(10))
print("-----")
np.random.seed(42)
print(np.random.sample(10))    # alias of random_sample
print("-----")
np.random.seed(42)
print(np.random.random(10))
print("-----")
np.random.seed(42)
print(np.random.rand(10))
```

```
[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864 0.15599452
 0.05808361 0.86617615 0.60111501 0.70807258]
```

```
[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864 0.15599452
 0.05808361 0.86617615 0.60111501 0.70807258]
```

```
[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864 0.15599452
 0.05808361 0.86617615 0.60111501 0.70807258]
```

```
[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864 0.15599452
 0.05808361 0.86617615 0.60111501 0.70807258]
```

```
np.random.seed(42)
print(np.random.random_sample((2, 3)))
print("-----")
np.random.seed(42)
print(np.random.sample((2, 3)))
print("-----")
np.random.seed(42)
print(np.random.random((2, 3)))
print("-----")
np.random.seed(42)
print(np.random.rand(2, 3))
```

```
[[0.37454012 0.95071431 0.73199394]
 [0.59865848 0.15601864 0.15599452]]
```

```
[[0.37454012 0.95071431 0.73199394]
 [0.59865848 0.15601864 0.15599452]]
```

```
[[0.37454012 0.95071431 0.73199394]
 [0.59865848 0.15601864 0.15599452]]
```

```
[[0.37454012 0.95071431 0.73199394]
 [0.59865848 0.15601864 0.15599452]]
```

```
items = [1, 2, 3, 4, 5, 6]
np.random.shuffle(items)
items
```

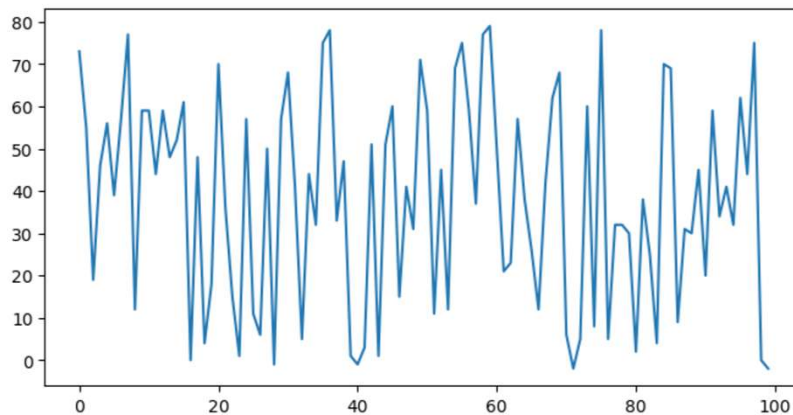
```
[5, 3, 1, 6, 2, 4]
```

not tuple

Basic plot types: Line plot

- Data points plotted as a line.
- Typically used when the x-axis is continuous or when it's acceptable to make such an assumption.
 - Example: time-series data
- Multiple lines can be combined for comparison.
 - Lines can have different styles.
 - Solid by default and can add different styles when plotting more than one line.

```
plt.figure(figsize=(8, 4))  
  
time = np.arange(0, 100)  
profit = np.random.randint(-2, 80, 100)  
plt.plot(time, profit)  
plt.show()
```

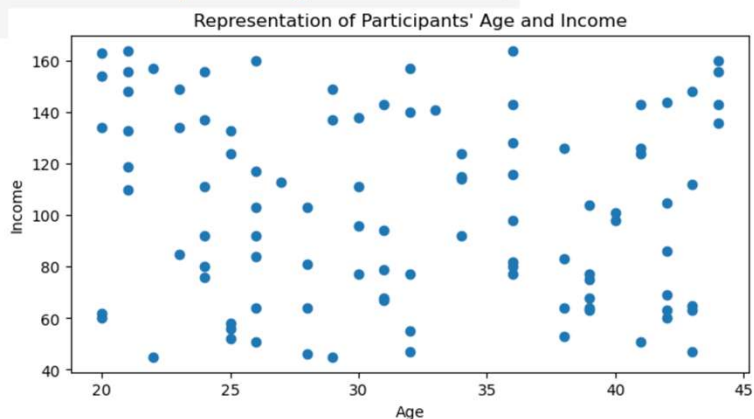


Basic plot types: Scatter plot

- Show multiple dimensions (typically two) of the data.
- Each mark is a data instance and is associated with an x and y value.
 - Data points plotted as dots, circles, or other symbols across two axes.
- Typically used when data represent multiple entities.
- Can include more than two dimensions.
 - Symbols can use different shapes to represent a third, typically categorical variable.
 - Symbols can use size or hue to represent a third continuous variable.
- More than three is typically not recommended.

```
age = np.random.randint(20, 45, 100)
income = np.random.randint(45, 165, 100)
```

```
plt.figure(figsize=(8, 4))
plt.scatter(age, income)
plt.xlabel("Age")
plt.ylabel("Income")
plt.title("Representation of Participants' Age and Income")
plt.show()
```

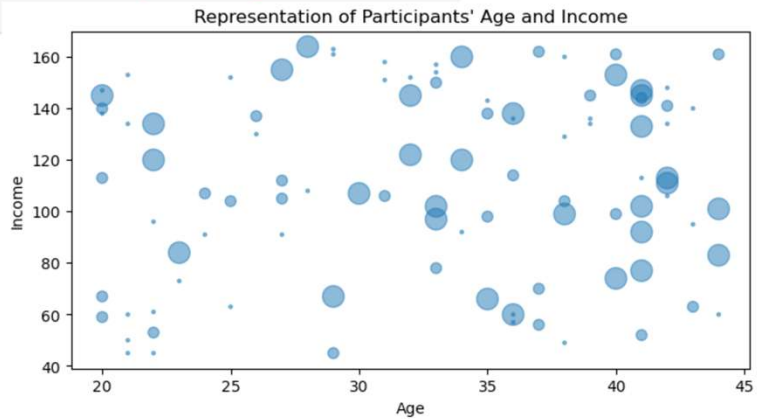


```

age = np.random.randint(20, 45, 100)
income = np.random.randint(45, 165, 100)
credit_score = np.random.choice([5, 50, 200], 100)

plt.figure(figsize=(8, 4))
plt.scatter(age, income, s=credit_score, alpha=0.5)
plt.xlabel("Age")
plt.ylabel("Income")
plt.title("Representation of Participants' Age and Income")
plt.show()

```



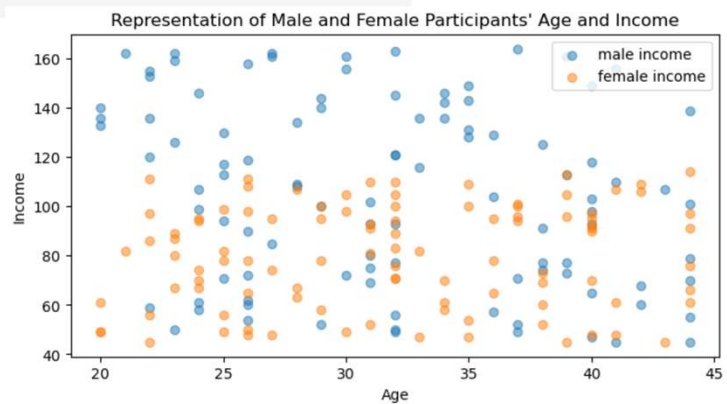
```

plt.figure(figsize=(8, 4))

age = np.random.randint(20, 45, 100)
income_male = np.random.randint(45, 165, 100)
income_female = np.random.randint(45, 115, 100)

plt.scatter(age, income_male, alpha=0.5, label="male income")
plt.scatter(age, income_female, alpha=0.5, label="female income")
plt.legend()
plt.xlabel("Age")
plt.ylabel("Income")
plt.title("Representation of Male and Female Participants' Age and Income")
plt.show()

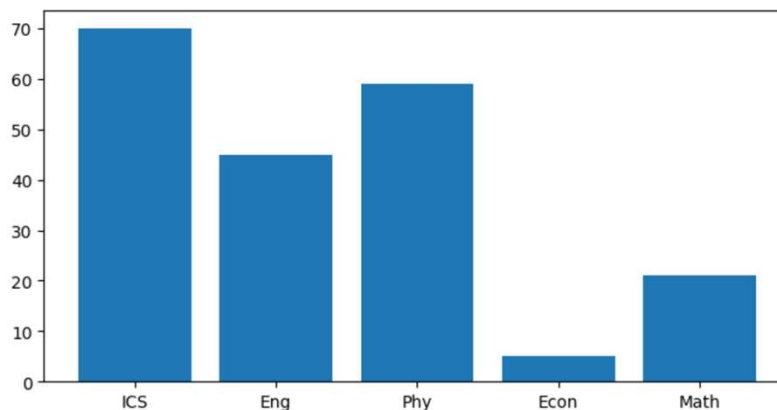
```



Basic plot types: Bar plot

- Shows the relationship between a numeric and a categorical variable.
 - Each unique categorical variable is associated with a bar, the length of which represents a numerical value.
 - E.g., number of students in ICS, Eng, Phys, Econ, and Math.
- Categorical variable can be further split across factors.
 - E.g., number of students in ICS, Eng, Phys, Econ, and Math split by gender.
 - Data can be shown side by side, stacked, etc.

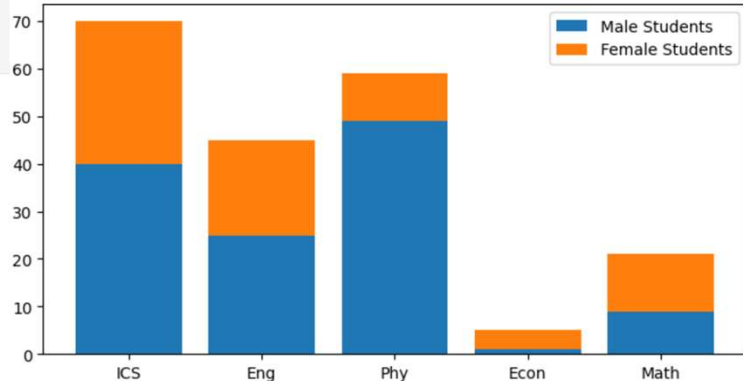
```
discipline = ["ICS", "Eng", "Phy", "Econ", "Math"]  
nb_students = [70, 45, 59, 5, 21]  
  
plt.figure(figsize=(8,4))  
plt.bar(discipline, nb_students)  
plt.show()
```



```
discipline = ["ICS", "Eng", "Phy", "Econ", "Math"]
nb_male_students = [40, 25, 49, 1, 9]
nb_female_students = [30, 20, 10, 4, 12]
```

```
x_values = np.arange(len(discipline))
```

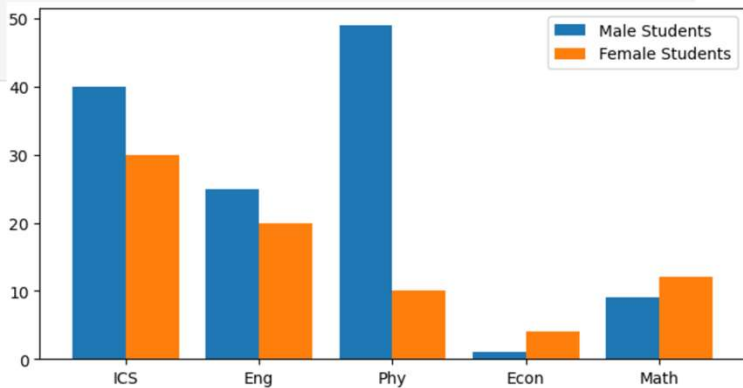
```
plt.figure(figsize=(8,4))
plt.bar(x_values, nb_male_students, label="Male Students")
plt.bar(x_values, nb_female_students, bottom=nb_male_students, label="Female Students")
plt.xticks(x_values, discipline)
plt.legend()
plt.show()
```



```
discipline = ["ICS", "Eng", "Phy", "Econ", "Math"]
nb_male_students = [40, 25, 49, 1, 9]
nb_female_students = [30, 20, 10, 4, 12]
```

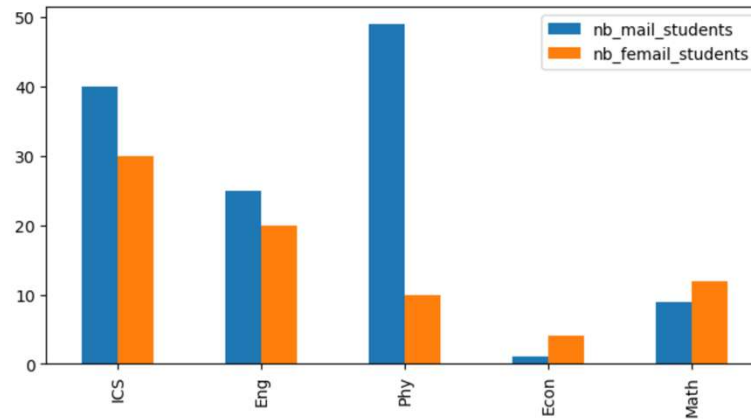
```
x_values = np.arange(len(discipline))
bar_width = 0.4
```

```
plt.figure(figsize=(8,4))
plt.bar(x_values - bar_width/2, nb_male_students, width=bar_width, label="Male Students")
plt.bar(x_values + bar_width/2, nb_female_students, width=bar_width, label="Female Students")
plt.xticks(x_values, discipline)
plt.legend()
plt.show()
```

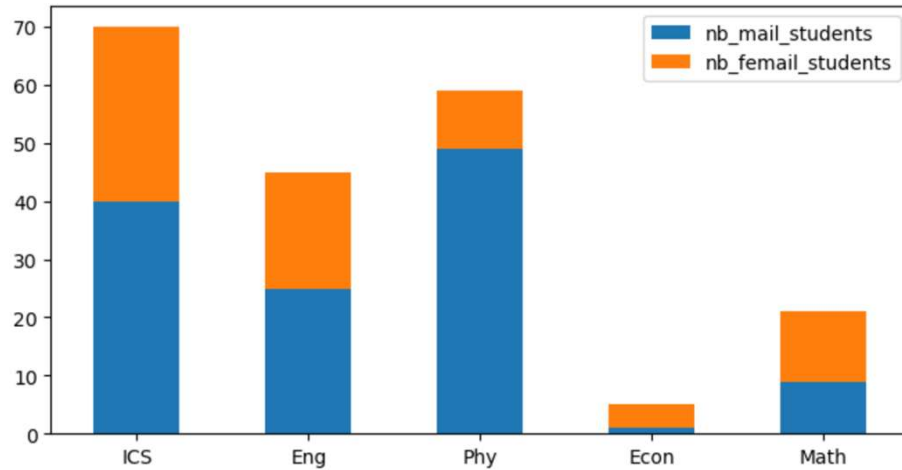


```
discipline = ["ICS", "Eng", "Phy", "Econ", "Math"]
df = pd.DataFrame({"nb_mail_students": [40, 25, 49, 1, 9],
                  "nb_femail_students": [30, 20, 10, 4, 12]},
                  index=discipline)
display(df)
df.plot.bar(figsize=(8, 4));
```

	nb_mail_students	nb_femail_students
ICS	40	30
Eng	25	20
Phy	49	10
Econ	1	4
Math	9	2



```
df.plot.bar(stacked=True, figsize=(8, 4), rot=0);
```



Basic plot types: Histogram

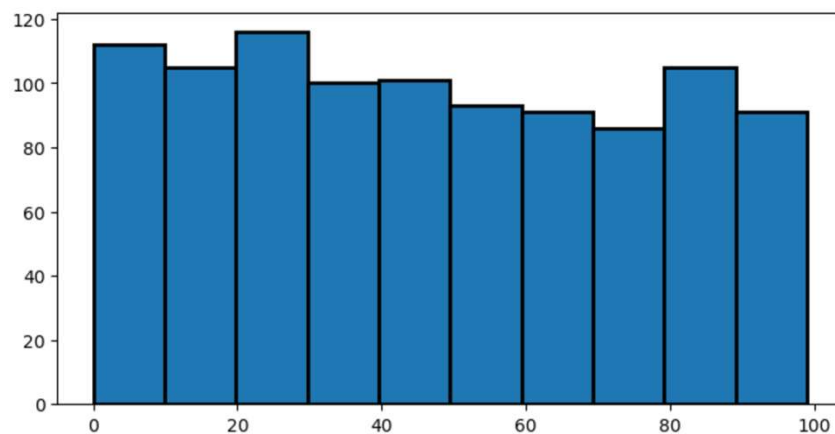
- A histogram visualizes the frequency of data points within bins (intervals), showing how data is distributed across a range of values.
- Useful for understanding the *distribution* of a dataset
 - Height of each bin approximates the frequency or probability of observations in that range.
- Typically used when:
 - The data is unidimensional (single variable)
 - Interested in the shape, spread, or concentration of values in the dataset
- The number of bins can be defined manually or computed automatically to best represent the data.

```
data = np.random.randint(0, 100, 1000)
```

```
plt.figure(figsize=(8,4))
```

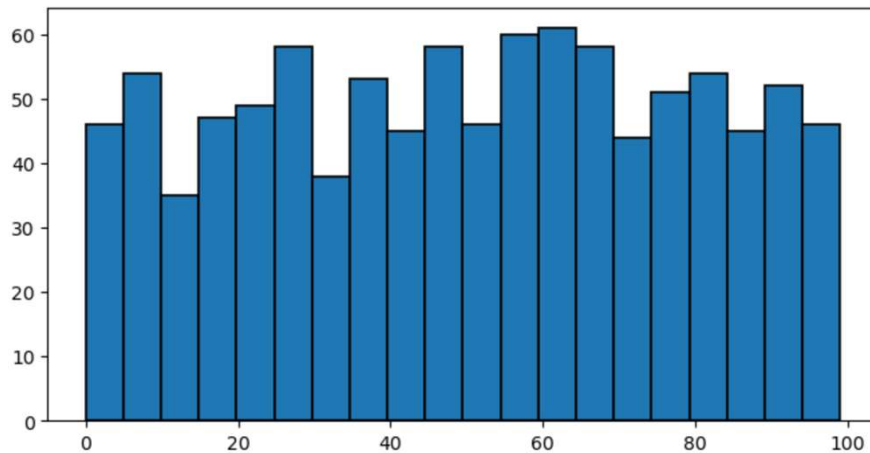
```
plt.hist(data, edgecolor='black', linewidth=2)
```

```
(array([112., 105., 116., 100., 101., 93., 91., 86., 105., 91.]),  
array([ 0. , 9.9, 19.8, 29.7, 39.6, 49.5, 59.4, 69.3, 79.2, 89.1, 99. ]),  
<BarContainer object of 10 artists>)
```



```
data = np.random.randint(0, 100, 1000)

plt.figure(figsize=(8, 4))
plt.hist(data, bins=20, edgecolor='black', linewidth=1.2)
plt.show()
```



Class exercise: Reproducing a plot

- The file `ex_vis.ipynb` is available on Lamaku. Please download it (and `ex_vis_figure.png`) and try the problem. If you have any questions, feel free to post them on Slack or contact us directly.
 - Note: Code submission and Slack posts are **NOT required**.